Programming Assignment II: Summary

CS683: Advanced Computer Architecture, Autumn 2026
Computer Science and Engineering
Indian Institute of Technology, Bombay

Team Member	Roll No
Tejeswar	24b0918
Bhavesh Ramakrishnan Karthik	24b0945
Sushant Padha	24b1057
Rhythm Kedia	24b1071

Contents

Introduction **3**

Implementation files . . . 3

Methodology . . . 3

Task 1: Stride Pattern Prefetching **4**

Task 1A: IP-stride prefetcher . . . 4

Task 1B: Design . . . 4

Storage overhead . . . 4

Accuracy throttling . . . 4

Results . . . 4

Task 2A: Instruction-Conscious L2 Replacement Policy (I-RRIP) **5**

Interface change . . . 5

Design: insertion, promotion, eviction . . . 5

Results . . . 8

Task 2B: Dynamic I-RRIP **10**

Leader sets and decision logic 10

Sensitivity study 10

Results 10

Task 3: Incorporating SPP with Dynamic I-RRIP **11**

Prefetch-aware Dynamic I-RRIP 11

Rationale 11

Design choices 12

Results 14

Introduction

Report from team flipflops.

Implementation files

Our code is on the `dev` branch of the repository.

```
prefetcher/spp.l1d_pref             Task 1B: Stride Pattern Prefetcher
replacement/irrip.l2c_repl          Task 2A: I-RRIP
replacement/dyn_irrip.l2c_repl      Tasks 2B and 3: Dynamic I-RRIP
```

Methodology

- Every provided trace is simulated with 25M warmup and 25M simulation instructions.
- The baseline has no L1D prefetcher and LRU replacement at L2C. Speedup is IPC divided by the baseline's IPC on the same trace.

Task 1

Stride Pattern Prefetching

Task 1A: IP-stride prefetcher

Task 1B: Design

Storage overhead

Accuracy throttling

Results

Task 2A

Instruction-Conscious L2 Replacement Policy (I-RRIP)

Interface change

We added a last parameter, `uint8_t isInstruction`, to `update_replacement_state`: in the member-function pointer type in `inc/cache.h`, in every cache level's definition, and at every call site. Each call in `src/cache.cc` passes the `instruction` field of the packet it handles: the MSHR entry on a fill from the lower level, the write queue entry on a writeback that misses, and the read, write or prefetch queue entry on a hit. The BTB calls in `src/ooo_cpu.cc` pass 0. The flag is also set on fills of page-table lines for instruction translations, so I-RRIP protects those lines too; the L2C instruction load MPKI does not count them, because it counts only load requests. I-RRIP stores the flag of each line when the line is filled, because `find_victim` does not receive it.

Design: insertion, promotion, eviction

An L1I miss stalls fetch; the out-of-order window can only keep executing instructions already fetched, so it hides little of the miss. A data miss is often overlapped with independent work in the window. An L2 hit is therefore worth more for an instruction line than for a data line, and I-RRIP gives up some data hits to keep instruction lines. It starts from SRRIP with 2-bit RRPVs (maximum RRPV 3), which inserts every line at RRPV 2, promotes every hit to RRPV 0, and evicts the first line at RRPV 3, aging all lines until one reaches 3.

	SRRIP	I-RRIP (final, level 8)
Insertion	every line at 2	instruction lines at 2 (unchanged); data lines at 3
Promotion	every hit to 0	instruction hit to 0 (unchanged); data hit: RRPV − 1
Eviction	first line at 3; else age all lines	first *data* line at 3, else any line at 3; else age: data lines every round, instruction lines every 4th round

Table 1: Replacement rules of SRRIP and the final I-RRIP.

What the eviction rule does. A data line needs at most three aging rounds to reach RRPV 3, and an instruction line is aged only in the fourth round. So in a set that holds at least one data line, the victim search ends before any instruction line is aged. An instruction line is then evicted only if it already has RRPV 3, which it can reach only by aging while its set holds no data line. **TODO: how often a set that holds a data line still evicts an instruction line is not measured.**

How we chose the rules. `irrip.l2c_repl` implements eight variants, which we call levels; the default is level 8, the final policy. Each level changes one rule of a previous one:

- Level 1: data-first eviction, instruction lines aged every 2nd round, every line inserted at 2, instruction hit to 0, data hit RRPV − 1.

- Level 2: level 1, but instruction lines inserted at 0.
- Level 3: level 1, plus a cap of 6 instruction lines per 8-way set: when a set holds 6 or more, the instruction line with the highest RRPV is evicted.
- Level 4: level 2, but data hits promoted to 1. Level 5: level 1, but data hits promoted to 1.
- Level 6: level 1, but instruction lines aged every 4th round.
- Level 7: level 1, but data lines inserted at 3.
- Level 8: levels 6 and 7 together.

		Load MPKI			MPKI ratio to SRRIP		
Policy	Speedup	L2C	Instr.	Data	L2C	Instr.	Data
SRRIP	1.0004	15.81	9.85	5.96	1.000	1.000	1.000
level 1	1.0182	13.55	6.35	7.20	0.887	0.656	1.173
level 2	1.0089	14.47	7.08	7.39	0.934	0.719	1.199
level 3	1.0329	14.99	8.20	6.79	0.979	0.881	1.125
level 4	1.0084	14.50	7.12	7.38	0.936	0.725	1.197
level 5	1.0178	13.56	6.37	7.19	0.888	0.660	1.172
level 6	1.0194	13.85	6.15	7.70	0.904	0.627	1.240
level 7	1.0216	13.73	5.81	7.92	0.901	0.597	1.277
level 8 (I-RRIP)	1.0216	13.91	5.76	8.14	0.911	0.590	1.306

Table 2: Every level, averaged over the three traces. Speedup is over LRU (geometric mean). MPKI columns are arithmetic means; ratio columns are each MPKI relative to SRRIP (geometric mean of the per-trace ratios). Percentages in the text compare the MPKI columns.

Insertion.

- *Instruction lines: not modified (RRPV 2).* Inserting them at 0 (level 2 vs. level 1) raises mean instruction MPKI by 11% and mean data MPKI by 3%, and lowers the speedup from 1.0182 to 1.0089. At RRPV 2 an instruction line needs one L2 hit to reach RRPV 0, so only instruction lines that are reused at the L2 get full protection. At RRPV 0, every instruction fill gets full protection, including lines that are not reused before they are evicted, and data-first eviction keeps them for a long time. **TODO: we have not measured how many instruction fills are never reused at the L2; this explanation is consistent with both MPKIs rising, but not proven.**
- *Data lines: modified (RRPV 3).* Level 7 vs. level 1 lowers instruction MPKI by 8.6% and raises data MPKI by 10%, and the speedup rises from 1.0182 to 1.0216. A data line inserted at 3 is a victim candidate ahead of every instruction line unless it is hit first, so data that is not reused soon cannot push instruction lines out. A hit moves it to RRPV 2; it is still evicted before any instruction line once it ages back to 3.

Promotion.

- *Instruction hits: not modified (RRPV 0).* A hit is the only evidence of reuse, and RRPV 0 gives a reused instruction line the most protection. We did not test other values. **TODO: no ablation of instruction promotion.**

- *Data hits: modified (RRPV − 1).* The data promotion rule barely matters: promoting data hits to 1 instead (level 5 vs. level 1, level 4 vs. level 2) changes the mean speedup by at most 0.05% and each mean MPKI by at most 0.05. **TODO: why the data promotion rule does not matter is not verified.**

Eviction.

- *Modified: data first, instruction lines aged more slowly.* Aging instruction lines every 4th round instead of every 2nd (level 6 vs. level 1) lowers instruction MPKI by 3.1% and raises data MPKI by 6.9%, for a speedup of 1.0194 instead of 1.0182. **TODO: we did not measure data-first eviction alone against SRRIP; its effect is measured only together with the other rules (level 1).**
- On top of data insertion at 3, slower aging adds little (level 8 vs. level 7): instruction MPKI falls by 0.8%, data MPKI rises by 2.8%, and the speedup is the same to four digits (1.0216). **TODO: how often the victim search reaches the aging step is not measured; the 2.8% data MPKI rise shows it is reached often enough to matter.** We keep level 8 because it has the lowest instruction MPKI, which is the goal of this task.

The instruction cap (level 3). Level 3 has the highest speedup (1.0329), but its instruction MPKI is 29% above level 1's, so it does not meet the goal of this task, and we did not choose it. The cap lowers LLC load misses (Table 3): on Trace1, from 301.9k with level 1 to 267.4k, while L2C load misses rise from 447.4k to 464.5k. **TODO: why the cap lowers LLC misses is not verified. Our hypothesis: the cap sends instruction lines back to the LLC more often, which keeps them recent in the LRU LLC. ChampSim does not split LLC statistics into instruction and data, so we cannot test it from the current logs. TODO: the cap value (6) comes from a sweep over 4 to 8 at shorter, non-final instruction counts; rerun it at the final settings or drop the claim.**

Trace	Policy	L2C load misses	LLC load hits	LLC load misses
Trace1	LRU	442.4	135.5	306.9
	level 1	447.4	145.5	301.9
	level 3	464.5	197.1	267.4
	level 8	465.0	166.2	298.8
Trace2	LRU	683.9	478.7	205.3
	level 1	474.8	266.0	208.8
	level 3	555.7	367.2	188.5
	level 8	481.2	275.7	205.5
Trace3	LRU	98.9	20.3	78.6
	level 1	94.2	17.9	76.3
	level 3	104.2	30.4	73.9
	level 8	96.7	21.7	75.0

Table 3: L2C load misses and LLC load hits and misses, in thousands, over the 25M simulated instructions.

Results

Trace	Policy	IPC	Speedup	L2C	L2C instr.	L2C data
Trace1	LRU (baseline)	0.2323	1.0000	17.70	10.34	7.36
	SRRIP	0.2322	0.9995	17.64	10.56	7.08
	I-RRIP	0.2339	1.0068	18.60	8.98	9.62
Trace2	LRU (baseline)	0.2512	1.0000	27.36	18.85	8.50
	SRRIP	0.2518	1.0026	25.75	17.78	7.96
	I-RRIP	0.2639	1.0509	19.25	7.63	11.62
Trace3	LRU (baseline)	0.3821	1.0000	3.95	1.12	2.83
	SRRIP	0.3817	0.9991	4.04	1.20	2.84
	I-RRIP	0.3850	1.0076	3.87	0.67	3.19
Mean	LRU (baseline)	0.2885	1.0000	16.34	10.11	6.23
	SRRIP	0.2886	1.0004	15.81	9.85	5.96
	I-RRIP	0.2943	1.0216	13.91	5.76	8.14

Table 4: IPC, speedup over LRU, and L2C load MPKI (total, instruction, data). L2C load MPKI is the sum of instruction and data load MPKI. Mean: geometric for speedup, arithmetic otherwise.

		Load MPKI			MPKI ratio to LRU		
Policy	Speedup	L2C	Instr.	Data	L2C	Instr.	Data
LRU (baseline)	1.0000	16.34	10.11	6.23	1.000	1.000	1.000
SRRIP	1.0004	15.81	9.85	5.96	0.986	1.009	0.967
I-RRIP	1.0216	13.91	5.76	8.14	0.898	0.595	1.263

Table 5: Means over the three traces (geometric for speedup, arithmetic for MPKI), and each MPKI relative to LRU (geometric mean of the per-trace ratios).

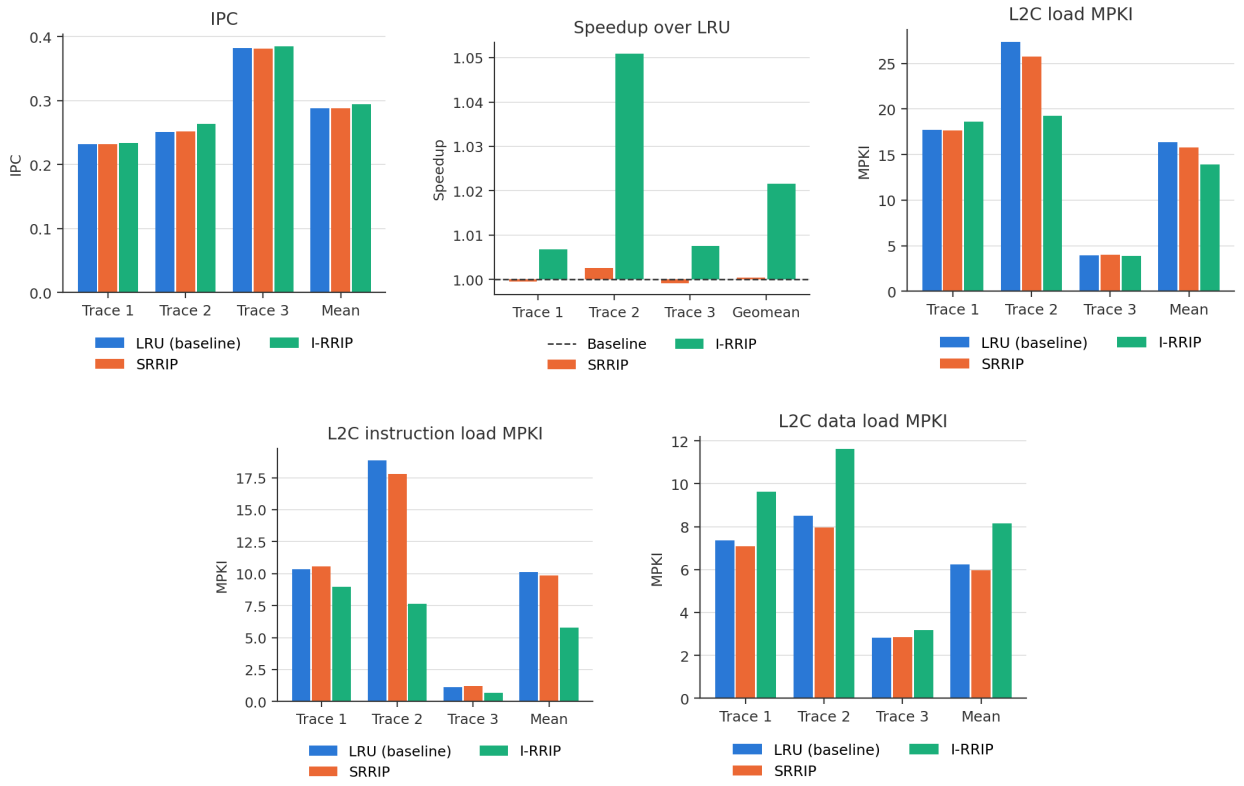


Figure 1: IPC, speedup over LRU (dashed line), and L2C load MPKI per trace.

I-RRIP lowers L2C instruction load MPKI on every trace: by 13% on Trace1, 60% on Trace2 and 40% on Trace3. Data load MPKI rises on every trace, by 31%, 37% and 13%. SRRIP alone is within 0.3% of LRU on every trace, so the gain comes from the instruction priority, not from RRIP.

Instruction misses cost more than data misses. On Trace1, I-RRIP has 5% *more* L2C load misses than LRU (18.60 vs. 17.70 MPKI), yet its IPC is 0.7% higher: it trades 1.35 MPKI of instruction misses for 2.26 MPKI of data misses. This is consistent with the argument above that an instruction miss stalls fetch while a data miss is partly overlapped.

Trace2 gains the most. The L1I MPKI, which the L2 policy does not change, is 14.5 on Trace1, 40.0 on Trace2 and 2.1 on Trace3. Trace2 sends the most instruction misses to the L2 and gains the most (5.1%). Trace1 and Trace3 gain about the same (0.7% and 0.8%) despite a 7× difference in L1I MPKI, so L1I MPKI alone does not predict the gain.

Why a 60% cut gives 5%. On Trace2, I-RRIP removes 202.7k L2C load misses relative to LRU. LLC load hits fall by 203.0k over the same run, and LLC load misses stay at 205.3k vs. 205.5k (Table 3). So the misses that I-RRIP removes were LLC hits, not DRAM accesses. Each one saves the difference between an LLC hit and an L2 hit (20 cycles: the LLC latency on top of the L2C latency in this configuration), not a DRAM access.

Task 2B

Dynamic I-RRIP

Leader sets and decision logic

Sensitivity study

Results

Task 3

Incorporating SPP with Dynamic I-RRIP

Prefetch-aware Dynamic I-RRIP

Dynamic I-RRIP (Task 2B) keeps 8 *leader* sets on plain RRIP. Every 256 leader accesses (prefetches and writebacks not counted) it computes the instruction share of their accesses and misses, and runs all other (*follower*) sets as I-RRIP if the share is above 0.15, else as RRIP. Task 3 adds one rule to the follower sets: **a prefetch never raises a line's priority.**

- *Prefetch fill:* inserted at RRPV 3, the first to be evicted (a demand fill goes in at RRPV 2 in RRIP mode).
- *Prefetch hit:* no promotion (Task 2B promoted it to RRPV 0 in RRIP mode, by one step in I-RRIP mode).

Demand and writeback hits promote as before. Leader sets stay plain RRIP, so the mode decision is still made on an unmodified policy. The rule reads only the packet type and needs no storage.

Rationale

The L2C copy of a prefetched line is idle. An SPP prefetch that misses the L1D fills the L2C and the L1D together. While the L1D holds the line, demands hit there, so the L2C copy can only be hit after the L1D evicts it. A prefetch that hits the L2C is filled into the L1D again, so the L2C copy is idle again. (When half the L1D MSHRs are busy, SPP fills only the L2C; these are 1.9–4.5% of the prefetches that reach the L2C.)

Under plain RRIP, prefetched lines are the least reused. We record for every L2C line what filled it and whether a demand hit it before eviction (Table 6). With every set on RRIP, 3.9–7.6% of prefetched lines get a demand hit, against 10.1–18.6% of demand data and 12.9–17.8% of instruction lines.

	Prefetch fill			Demand data			Instruction		
L2C policy	T1	T2	T3	T1	T2	T3	T1	T2	T3
RRIP	7.6	3.9	5.7	18.6	15.9	10.1	12.9	14.6	17.8
RRIP + rule	4.0	2.0	2.8	18.9	15.9	10.3	13.0	14.6	18.4
I-RRIP	3.3	2.2	2.5	3.4	4.7	2.3	14.7	20.1	24.0
I-RRIP run, leader sets	8.4	4.9	5.2	17.4	16.3	13.6	13.1	13.2	19.0
Dyn. I-RRIP + rule	3.2	2.3	2.3	3.4	5.2	2.3	14.7	21.5	26.3

Table 6: Percentage of evicted L2C lines that got at least one demand hit before eviction, by what filled them, with SPP at the L1D (25M simulated instructions). RRIP, I-RRIP: follower sets pinned to that mode. Rule: the prefetch rule. Leader sets always run RRIP; they evict only 100–300 prefetched lines per trace, so that row's prefetch column is rough. T1–T3: Traces 1–3.

Deadness measured under I-RRIP is inflated. I-RRIP inserts every data line at RRPV 3 and evicts data before instructions, so data lines die young and evict each other, while instruction lines stay. Demand data reuse drops from 10.1–18.6% under RRIP to 2.3–4.7% under I-RRIP, yet the leader sets of the same I-RRIP runs, which stay on RRIP, still reuse 13.6–17.4% of their data: the lines are not dead by nature, I-RRIP kills them. Table 7 shows the shift: I-RRIP evicts more data lines and up to 2.3× fewer instruction lines, and instruction reuse rises to 14.7–24.0%. Prefetched lines are inflated the same way (2.2–3.3% reused under I-RRIP). So the case for the rule rests on the RRIP numbers above, not on the I-RRIP ones.

Victim filled by	Policy	T1	T2	T3
Demand data	RRIP	238	263	86
	I-RRIP	307	378	96
Instruction	RRIP	265	432	33
	I-RRIP	225	192	18
All	RRIP	618	789	171
	I-RRIP	776	793	218

Table 7: L2C evictions (thousands, all sets) by what filled the victim, follower sets pinned to RRIP or I-RRIP, SPP on, prefetch rule off.

Where the rule acts. In I-RRIP mode a prefetch fill is already a data fill at RRPV 3, so only the hit rule changes anything. In RRIP mode both parts are new. The followers spend 86–100% of the epochs in I-RRIP mode, so the rule can only have a small effect on the dynamic policy.

Design choices

We ran each change with all follower sets pinned to one mode, to separate it from the mode decision (Table 8).

- *Prefetch fill at RRPV 3:* no change in I-RRIP mode; in RRIP mode it lowers L2C load MPKI by 0.7% and instruction load MPKI by 0.7%. Kept.

- *No promotion on prefetch hits:* lowers L2C load MPKI by 0.2% in I-RRIP mode; with the fill rule, by 0.7% (instruction 0.8%) in RRIP mode. Kept.
- *Evict unused prefetched lines first* among lines at RRPV 3: about the same MPKI reduction, but it needs a prefetch bit per line. Dropped.
- *Adaptive counter:*
- *Bypass of prefetch fills at the L2C:*

Across all combinations the speedup moves by at most 0.05%: the misses saved are under 1% of the L2C misses. We kept the two changes that need no storage and lower misses in both modes.

					Load MPKI		Ratio to first row	
Mode	Hit	Insert	Evict	Speedup	L2C	Instr.	L2C	Instr.
I-RRIP	0	–	0	1.0393	12.99	5.78	1.000	1.000
	1	–	0	1.0397	12.99	5.78	0.998	1.000
	0	–	1	1.0397	12.99	5.78	0.998	1.000
	1	–	1	1.0398	12.98	5.78	0.998	0.999
RRIP	0	0	0	1.0177	14.90	9.75	1.000	1.000
	1	0	0	1.0176	14.87	9.72	0.998	0.998
	0	1	0	1.0180	14.83	9.69	0.993	0.993
	1	1	0	1.0179	14.83	9.68	0.993	0.992
	0	0	1	1.0180	14.83	9.68	0.994	0.992
	1	0	1	1.0178	14.82	9.66	0.994	0.990
	0	1	1	1.0179	14.82	9.68	0.994	0.991
	1	1	1	1.0178	14.81	9.67	0.993	0.990

Table 8: SPP + Dynamic I-RRIP with follower sets pinned to I-RRIP or RRIP mode. 1 = change on. Hit: no promotion on a prefetch hit. Insert: prefetch fill at RRPV 3 (data is inserted at RRPV 3 in I-RRIP mode anyway). Evict: evict an unused prefetched line first among lines at RRPV 3. Speedup: geometric mean over traces; MPKI: arithmetic mean; ratios: geometric mean of per-trace ratios to the first row of each mode. Final policy: Hit = 1, Insert = 1, Evict = 0.

Results

Trace	Config	IPC	Speedup	Accuracy (%)	Coverage (%)
Trace1	SPP	0.2347	1.0099	55.9	11.4
	Dyn. I-RRIP	0.2339	1.0067	–	–
	SPP + Dyn. (unaware)	0.2367	1.0187	55.7	11.4
	SPP + Dyn. (aware)	0.2368	1.0190	55.6	11.2
Trace2	SPP	0.2549	1.0150	64.4	4.7
	Dyn. I-RRIP	0.2629	1.0468	–	–
	SPP + Dyn. (unaware)	0.2664	1.0606	63.4	4.7
	SPP + Dyn. (aware)	0.2665	1.0611	63.6	4.7
Trace3	SPP	0.3917	1.0252	49.7	20.4
	Dyn. I-RRIP	0.3837	1.0044	–	–
	SPP + Dyn. (unaware)	0.3949	1.0337	50.1	20.4
	SPP + Dyn. (aware)	0.3946	1.0329	50.0	20.3
Mean	SPP	0.2938	1.0167	56.7	12.2
	Dyn. I-RRIP	0.2935	1.0191	–	–
	SPP + Dyn. (unaware)	0.2993	1.0375	56.4	12.1
	SPP + Dyn. (aware)	0.2993	1.0375	56.4	12.1

Table 9: IPC, speedup over the baseline (no L1D prefetcher, LRU at L2C), and L1D prefetch accuracy and coverage. Accuracy: useful prefetches over prefetches sent to the L2C; coverage: fraction of baseline L1D load misses removed. Aware: the final policy. Unaware: the same policy with the rule off. Mean: geometric for speedup, arithmetic otherwise.

Trace	Config	L1D	L2C	L2C instr.	L2C data
Trace1	SPP	9.47	16.74	10.39	6.35
	Dyn. I-RRIP	10.68	18.59	8.99	9.60
	SPP + Dyn. (unaware)	9.47	17.34	8.99	8.35
	SPP + Dyn. (aware)	9.48	17.33	8.98	8.35
Trace2	SPP	12.90	26.80	18.90	7.90
	Dyn. I-RRIP	13.55	19.23	7.94	11.30
	SPP + Dyn. (unaware)	12.91	18.70	8.03	10.67
	SPP + Dyn. (aware)	12.91	18.66	7.97	10.68
Trace3	SPP	2.71	3.12	1.15	1.97
	Dyn. I-RRIP	3.40	3.90	0.77	3.13
	SPP + Dyn. (unaware)	2.71	2.98	0.68	2.31
	SPP + Dyn. (aware)	2.71	2.99	0.68	2.32
Mean	SPP	8.36	15.55	10.15	5.41
	Dyn. I-RRIP	9.21	13.91	5.90	8.01
	SPP + Dyn. (unaware)	8.36	13.01	5.90	7.11
	SPP + Dyn. (aware)	8.37	12.99	5.88	7.12

Table 10: Load MPKI of the L1D, the L2C, and the L2C split into instruction and data loads.

	Load MPKI				Ratio to baseline			
Config	L1D	L2C	Instr.	Data	L1D	L2C	Instr.	Data
Baseline	9.21	16.34	10.11	6.23	1.000	1.000	1.000	1.000
SPP	8.36	15.55	10.15	5.41	0.876	0.901	1.009	0.823
Dyn. I-RRIP	9.21	13.91	5.90	8.01	1.000	0.900	0.631	1.242
SPP + Dyn. (unaware)	8.36	13.01	5.90	7.11	0.876	0.797	0.607	1.051
SPP + Dyn. (aware)	8.37	12.99	5.88	7.12	0.877	0.797	0.605	1.053

Table 11: Mean load MPKI (arithmetic), and each MPKI relative to the baseline (geometric mean of the per-trace ratios).

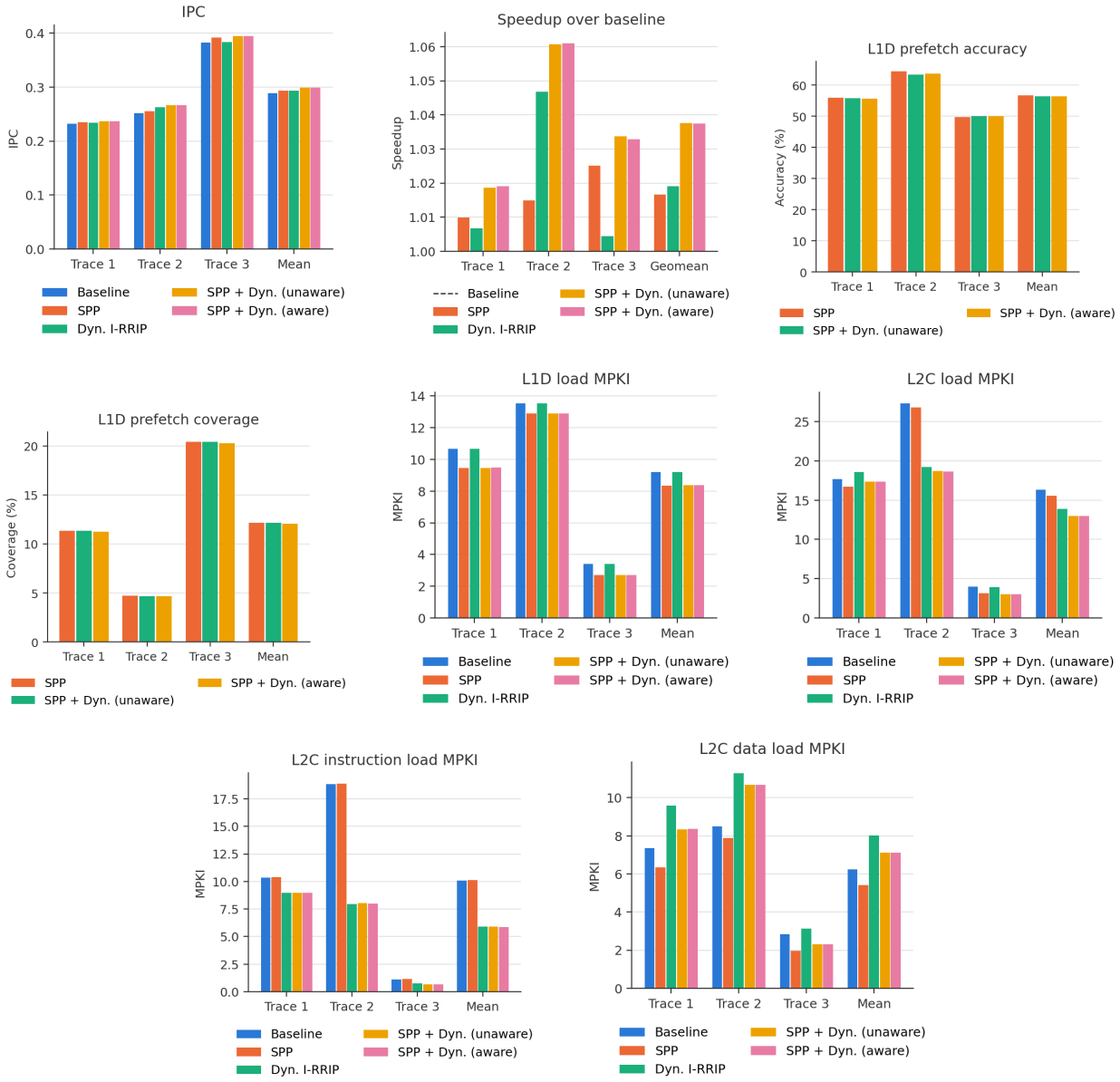


Figure 2: IPC, speedup over the baseline (dashed line), L1D prefetch accuracy and coverage, and load MPKI per trace.

SPP + Dynamic I-RRIP beats both on every trace. Mean speedup 1.0375, against 1.0167 for SPP and 1.0191 for Dynamic I-RRIP. Per trace it gains 1.90%, 6.11% and 3.29%; the best single component gains 0.99% (SPP), 4.68% (Dynamic I-RRIP) and 2.52% (SPP).

Why the gains add up. The two remove different misses. SPP removes L1D data misses (L1D load MPKI 0.88 of the baseline, with or without Dynamic I-RRIP, since the L2C policy does not change what the L1D holds). Dynamic I-RRIP removes L2C instruction misses (0.63 of the baseline alone, 0.60 combined); its extra L2C data misses (1.24 of the baseline alone) are mostly paid back by SPP (1.05 combined). The speedup is close to the product of the two ($1.0167 \times 1.0191 = 1.0361$).

What the rule adds. Little IPC. Accuracy and coverage are measured at the L1D and do not

change (56.4% and 12.1%). The mean speedup is 1.0375 with and without the rule; per trace the difference is +0.03%, +0.05% and −0.08%. Trace2, with the most L2C instruction misses, gets fewer of them (8.03 to 7.97 MPKI). On Trace3 the rule has 0.3% more L2C misses, although it lowers Trace3's misses in both pinned modes; both runs make the same 7 mode decisions, and we did not find the cause. The rule is small because the followers are mostly in I-RRIP mode, which already evicts prefetched lines first.